

AG2 - Actividad Guiada 2

Nombre: Reda Bennor Haliba

Link: https://colab.research.google.com/drive/19Pdr42jot9vFtna3p_HIUsKYiiBRMrnf?usp=sharing

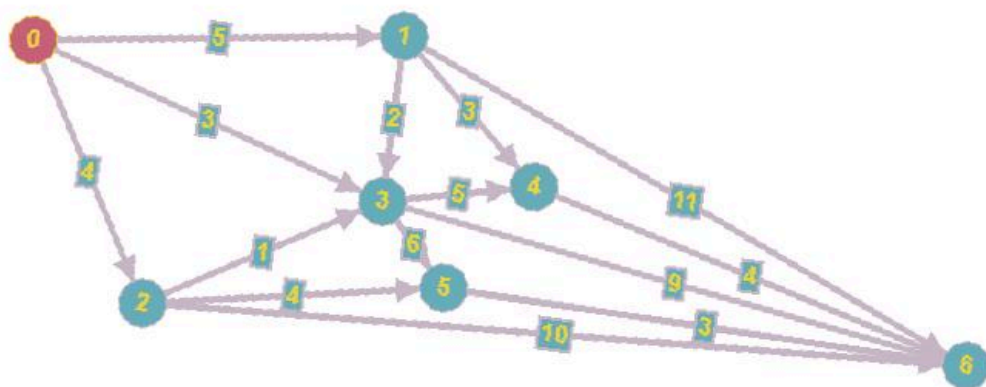
```
In [2]: import math
```

Programación Dinámica. Viaje por el río

- **Definición:** Es posible dividir el problema en subproblemas más pequeños, guardando las soluciones para ser utilizadas más adelante.
- **Características** que permiten identificar problemas aplicables:
 - Es posible almacenar soluciones de los subproblemas para ser utilizados más adelante
 - Debe verificar el principio de optimalidad de Bellman: "en una secuencia optima de decisiones, toda sub-secuencia también es óptima" (*)
 - La necesidad de guardar la información acerca de las soluciones parciales unido a la recursividad provoca la necesidad de preocuparnos por la complejidad espacial (cuantos recursos de espacio usaremos)

Problema

En un río hay n embarcaderos y debemos desplazarnos río abajo desde un embarcadero a otro. Cada embarcadero tiene precios diferentes para ir de un embarcadero a otro situado más abajo. Para ir del embarcadero i al j , puede ocurrir que sea más barato hacer un trasbordo por un embarcadero intermedio k . El problema consiste en determinar la combinación más barata.



*Consideramos una tabla $TARIFAS(i,j)$ para almacenar todos los precios que nos ofrecen los embarcaderos.

*Si no es posible ir desde i a j daremos un valor alto para garantizar que ese trayecto no se va a elegir en la ruta óptima(modelado habitual para restricciones)

```
In [3]: #Viaje por el rio - Programación dinámica
#####

TARIFAS = [
[0,5,4,3,float("inf"),999,999], #desde nodo 0
[999,0,999,2,3,999,11], #desde nodo 1
[999,999,0,1,999,4,10], #desde nodo 2
[999,999,999,0,5,6,9],
[999,999,999,999,0,999,4],
[999,999,999,999,999,0,3],
[999,999,999,999,999,999,0]
]

#999 se puede sustituir por float("inf") del modulo math
TARIFAS
```

```
Out[3]: [[0, 5, 4, 3, inf, 999, 999],
[999, 0, 999, 2, 3, 999, 11],
[999, 999, 0, 1, 999, 4, 10],
[999, 999, 999, 0, 5, 6, 9],
[999, 999, 999, 999, 0, 999, 4],
[999, 999, 999, 999, 999, 0, 3],
[999, 999, 999, 999, 999, 999, 0]]
```

```
In [4]: #Calculo de la matriz de PRECIOS y RUTAS
# PRECIOS - contiene la matriz del mejor precio para ir de un nodo a otro
# RUTAS - contiene los nodos intermedios para ir de un nodo a otro
#####
def Precios(TARIFAS):
#####
#Total de Nodos
N = len(TARIFAS[0])

#Inicialización de la tabla de precios
PRECIOS = [ [9999]*N for i in range(N)] #n x n
RUTA = [ [""]*N for i in range(N)]

#Se recorren todos los nodos con dos bucles(origen - destino)
# para ir construyendo la matriz de PRECIOS
for i in range(N-1):
    for j in range(i+1, N):
        MIN = TARIFAS[i][j]
        RUTA[i][j] = i

        for k in range(i, j):
            if PRECIOS[i][k] + TARIFAS[k][j] < MIN:
                MIN = min(MIN, PRECIOS[i][k] + TARIFAS[k][j] )
                RUTA[i][j] = k
        PRECIOS[i][j] = MIN
    return PRECIOS,RUTA
```

```
In [5]: PRECIOS,RUTA = Precios(TARIFAS)
#print(PRECIOS[0][6])

print("PRECIOS")
for i in range(len(TARIFAS)):
```

```

print(PRECIOS[i])

print("\nRUTA")
for i in range(len(TARIFAS)):
    print(RUTA[i])

```

```

PRECIOS
[9999, 5, 4, 3, 8, 8, 11]
[9999, 9999, 999, 2, 3, 8, 7]
[9999, 9999, 9999, 1, 6, 4, 7]
[9999, 9999, 9999, 9999, 5, 6, 9]
[9999, 9999, 9999, 9999, 9999, 999, 4]
[9999, 9999, 9999, 9999, 9999, 9999, 3]
[9999, 9999, 9999, 9999, 9999, 9999, 9999]

```

```

RUTA
['', 0, 0, 0, 1, 2, 5]
['', '', 1, 1, 1, 3, 4]
['', '', '', 2, 3, 2, 5]
['', '', '', '', 3, 3, 3]
['', '', '', '', '', 4, 4]
['', '', '', '', '', '', 5]
['', '', '', '', '', '', '']

```

```

In [6]: #Calculo de la ruta usando la matriz RUTA
def calcular_ruta(RUTA, desde, hasta):
    if desde == RUTA[desde][hasta]:
        #if desde == hasta:
            #print("Ir a :" + str(desde))
            return desde
    else:
        return str(calcular_ruta(RUTA, desde, RUTA[desde][hasta])) + ',' + str(RUTA

print("\nLa ruta es:")
calcular_ruta(RUTA, 0,6)

```

La ruta es:

```
Out[6]: '0,2,5'
```

Problema de Asignacion de tarea

```

In [7]: #Asignacion de tareas - Ramificación y Poda
#####
#   T A R E A
#   A
#   G
#   E
#   N
#   T
#   E

COSTES=[[11,12,18,40],
        [14,15,13,22],
        [11,17,19,23],
        [17,14,20,28]]

```

In [8]: *#Calculo del valor de una solucion parcial*

```
def valor(S,COSTES):  
    VALOR = 0  
    for i in range(len(S)):  
        VALOR += COSTES[S[i]][i]  
    return VALOR
```

```
valor((3,2, ),COSTES)
```

Out[8]: 34

In [9]: *#Coste inferior para soluciones parciales*

(1,3,) Se asigna la tarea 1 al agente 0 y la tarea 3 al agente 1

```
def CI(S,COSTES):  
    VALOR = 0  
    #Valores establecidos  
    for i in range(len(S)):  
        VALOR += COSTES[i][S[i]]  
  
    #Estimacion  
    for i in range( len(S), len(COSTES) ):  
        VALOR += min( [ COSTES[j][i] for j in range(len(S), len(COSTES)) ] )  
    return VALOR
```

```
def CS(S,COSTES):  
    VALOR = 0  
    #Valores establecidos  
    for i in range(len(S)):  
        VALOR += COSTES[i][S[i]]  
  
    #Estimacion  
    for i in range( len(S), len(COSTES) ):  
        VALOR += max( [ COSTES[j][i] for j in range(len(S), len(COSTES)) ] )  
    return VALOR
```

```
CI((0,1),COSTES)
```

Out[9]: 68

In [10]: *#Genera tantos hijos como posibilidades haya para la siguiente elemento de*
#(0,) -> (0,1), (0,2), (0,3)

```
def crear_hijos(NODO, N):  
    HIJOS = []  
    for i in range(N):  
        if i not in NODO:  
            HIJOS.append({'s':NODO +(i, )})  
    return HIJOS
```

In [11]: crear_hijos((0,) , 4)

Out[11]: [{'s': (0, 1)}, {'s': (0, 2)}, {'s': (0, 3)}]

In [12]: **def** ramificacion_y_poda(COSTES):
#Construccion iterativa de soluciones(arbol). En cada etapa asignamos un agente(
#Nodos del grafo { s:(1,2),CI:3,CS:5 }

```

#print(COSTES)
DIMENSION = len(COSTES)
MEJOR_SOLUCION=tuple( i for i in range(len(COSTES)) )
CotaSup = valor(MEJOR_SOLUCION,COSTES)
#print("Cota Superior:", CotaSup)

NODOS=[]
NODOS.append({'s':(), 'ci':CI((),COSTES) } )

iteracion = 0

while( len(NODOS) > 0):
    iteracion +=1

    nodo_prometedor = [ min(NODOS, key=lambda x:x['ci']) ][0]['s']
    #print("Nodo prometedor:", nodo_prometedor)

    #Ramificacion
    #Se generan Los hijos
    HIJOS = [ {'s':x['s'], 'ci':CI(x['s'], COSTES) } for x in crear_hijos(nodo_

    #Revisamos la cota superior y nos quedamos con la mejor solucion si llegamos
    NODO_FINAL = [x for x in HIJOS if len(x['s']) == DIMENSION ]
    if len(NODO_FINAL) >0:
        #print("\n*****Soluciones:", [x for x in HIJOS if len(x['s']) == DIMEN
        if NODO_FINAL[0]['ci'] < CotaSup:
            CotaSup = NODO_FINAL[0]['ci']
            MEJOR_SOLUCION = NODO_FINAL

    #Poda
    HIJOS = [x for x in HIJOS if x['ci'] < CotaSup ]

    #Añadimos Los hijos
    NODOS.extend(HIJOS)

    #Eliminamos el nodo ramificado
    NODOS = [ x for x in NODOS if x['s'] != nodo_prometedor ]

print("La solucion final es:" ,MEJOR_SOLUCION , " en " , iteracion , " iteraci

ramificacion_y_poda(COSTES)

```

La solucion final es: [{'s': (1, 2, 0, 3), 'ci': 64}] en 10 iteraciones para dimension: 4

Descenso del gradiente

```

In [13]: import math                                #Funciones matematicas
import matplotlib.pyplot as plt                    #Generacion de gráficos (otra opcion seaborn)
import numpy as np                                #Tratamiento matriz N-dimensionales y otras (fu
import scipy as sc

import random

```

Vamos a buscar el minimo de la funcion paraboloide :

$$f(x) = x^2 + y^2$$

Obviamente se encuentra en $(x,y)=(0,0)$ pero probaremos como llegamos a él a través del descenso del gradiente.

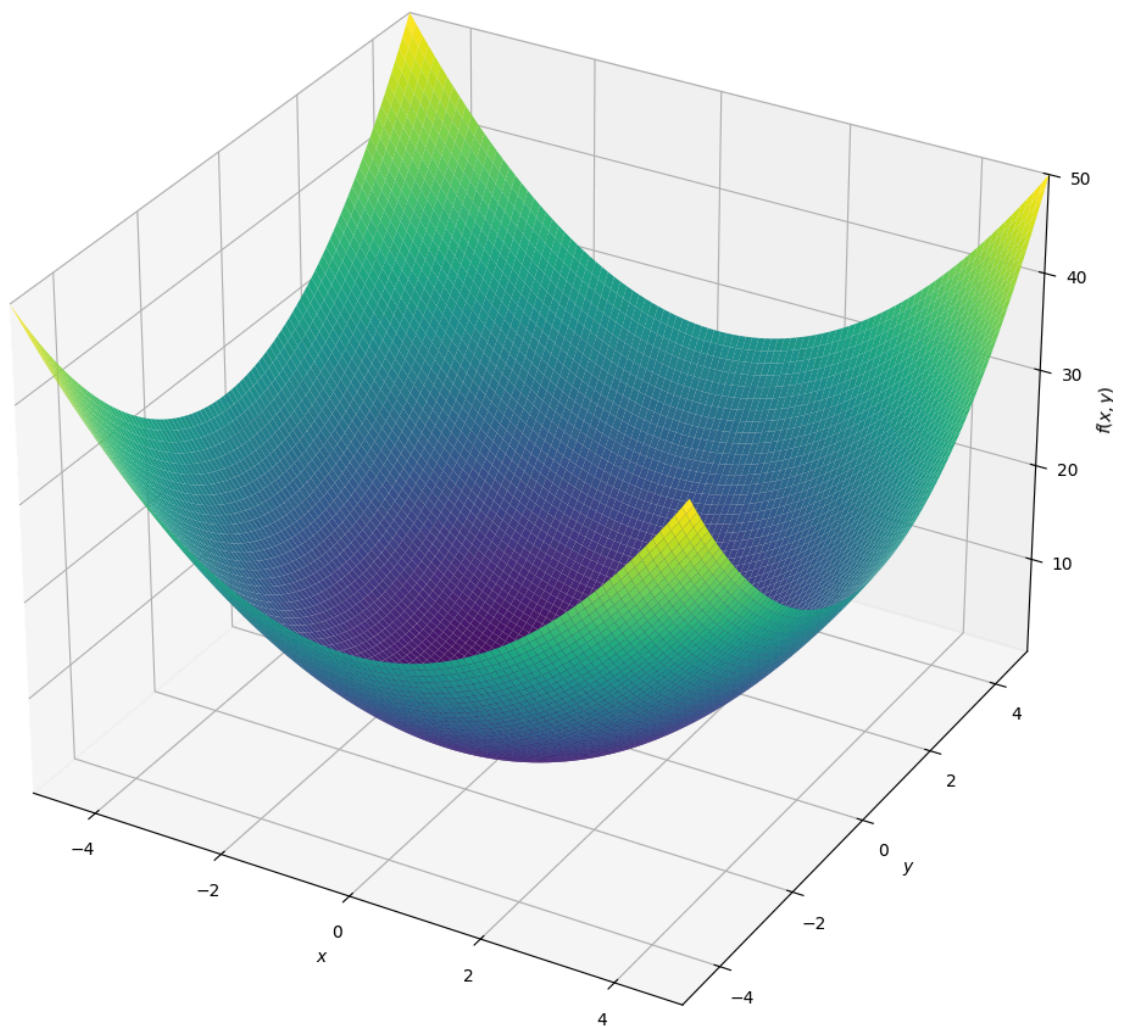
```
In [14]: #Definimos la funcion
#Paraboloide
f = lambda X:      X[0]**2 + X[1]**2      #Funcion
df = lambda X: [2*X[0] , 2*X[1]]         #Gradiente

df([1,2])
```

Out[14]: [2, 4]

```
In [15]: from sympy import symbols
from sympy.plotting import plot
from sympy.plotting import plot3d
x,y = symbols('x y')
plot3d(x**2 + y**2,
        (x,-5,5),(y,-5,5),
        title='x**2 + y**2',
        size=(10,10))
```

$x^2 + y^2$



```
Out[15]: <sympy.plotting.backends.matplotlibbackend.matplotlib.MatplotlibBackend at 0x791b89f8db50>
```

```
In [16]: #Prepara los datos para dibujar mapa de niveles de Z
resolucion = 100
rango=5.5

X=np.linspace(-rango,rango,resolucion)
Y=np.linspace(-rango,rango,resolucion)
Z=np.zeros((resolucion,resolucion))
for ix,x in enumerate(X):
    for iy,y in enumerate(Y):
        Z[iy,ix] = f([x,y])

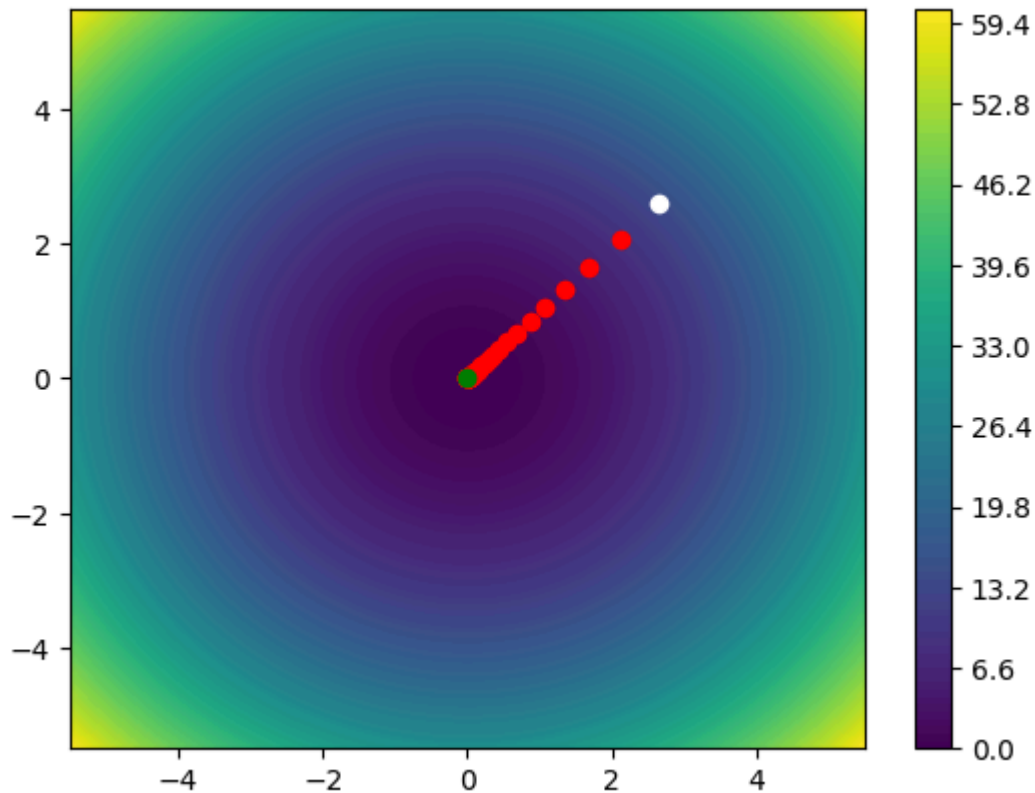
#Pinta el mapa de niveles de Z
plt.contourf(X,Y,Z,resolucion)
plt.colorbar()

#Generamos un punto aleatorio inicial y pintamos de blanco
P=[random.uniform(-5,5 ),random.uniform(-5,5 ) ]
plt.plot(P[0],P[1],"o",c="white")

#Tasa de aprendizaje. Fija. Sería más efectivo reducirlo a medida que nos acerca
TA=.1

#Iteraciones:50
for _ in range(50):
    grad = df(P)
    #print(P,grad)
    P[0],P[1] = P[0] - TA*grad[0] , P[1] - TA*grad[1]
    plt.plot(P[0],P[1],"o",c="red")

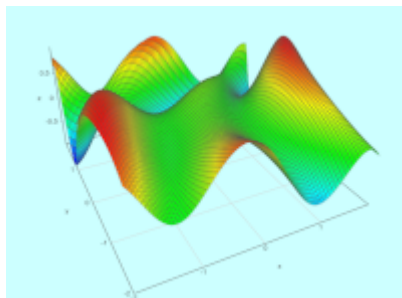
#Dibujamos el punto final y pintamos de verde
plt.plot(P[0],P[1],"o",c="green")
plt.show()
print("Solucion:" , P , f(P))
```



Solucion: [3.762455840240114e-05, 3.702313254385606e-05] 2.786319738335628e-09

¿Te atreves a optimizar la función?:

$$f(x) = \sin(1/2 * x^2 - 1/4 * y^2 + 3) * \cos(2 * x + 1 - e^y)$$



In []:

Solución: Descenso del Gradiente sobre la función propuesta

Se aplica el algoritmo de **descenso del gradiente** siguiendo la misma metodología vista en clase con el paraboloide, pero adaptada a la función más compleja:

$$f(x, y) = \sin\left(\frac{1}{2}x^2 - \frac{1}{4}y^2 + 3\right) \cdot \cos(2x + 1 - e^y)$$

Cálculo del gradiente

Para obtener las derivadas parciales se aplica la **regla del producto** combinada con la **regla de la cadena**:

$$\frac{\partial f}{\partial x} = \cos\left(\frac{1}{2}x^2 - \frac{1}{4}y^2 + 3\right) \cdot x \cdot \cos(2x + 1 - e^y) - \sin\left(\frac{1}{2}x^2 - \frac{1}{4}y^2 + 3\right) \cdot \sin(2x + 1 - e^y)$$

$$\frac{\partial f}{\partial y} = \cos\left(\frac{1}{2}x^2 - \frac{1}{4}y^2 + 3\right) \cdot \left(-\frac{y}{2}\right) \cdot \cos(2x + 1 - e^y) + \sin\left(\frac{1}{2}x^2 - \frac{1}{4}y^2 + 3\right) \cdot \sin(2x + 1 - e^y)$$

Parámetros elegidos

Parámetro	Paraboloide (clase)	Esta función
Tasa de aprendizaje	0.1	0.05
Iteraciones	50	500

Se reduce la tasa y se aumentan las iteraciones porque la superficie de esta función es mucho más irregular que un paraboloide, lo que requiere pasos más pequeños y más tiempo para converger.

Visualización

Se generan dos vistas complementarias:

1. **Mapa de contorno 2D** con la trayectoria del descenso (inicio en blanco, camino en rojo, solución en verde).
2. **Superficie 3D** con SymPy para apreciar la forma de la función.

Limitación importante

A diferencia del paraboloide $f(x, y) = x^2 + y^2$, que es **convexo** y tiene un único mínimo global, esta función **no es convexa** y presenta múltiples mínimos locales. Esto significa que, dependiendo del punto de inicio aleatorio, el algoritmo puede converger a diferentes mínimos locales sin garantía de alcanzar el mínimo global. Para abordar este problema se podrían emplear estrategias como ejecutar el algoritmo desde múltiples puntos de inicio y quedarse con la mejor solución.

```
In [21]: import math
import matplotlib.pyplot as plt
import numpy as np
import random
from sympy import symbols
from sympy.plotting import plot3d
from sympy import sin, cos, exp

# --- CONFIGURACIÓN GLOBAL ---
# Definimos los parámetros una sola vez para que ambos gráficos coincidan
rango = 2.5
resolucion = 100
TA = 0.05 # Tasa de aprendizaje
iteraciones = 500
# Punto de inicio aleatorio (Lo guardamos para usarlo en ambos si quisiéramos,
# aunque el gráfico 3D de sympy no suele mostrar el camino, solo la superficie)
P_inicial = [random.uniform(-2, 2), random.uniform(-2, 2)]

# --- DEFINICIÓN DE FUNCIONES ---
# Función matemática para evaluación numérica
```

```

f = lambda X: math.sin(1/2 * X[0]**2 - 1/4 * X[1]**2 + 3) * math.cos(2*X[0] + 1

# Gradiente (Derivadas parciales)
def df(X):
    x, y = X[0], X[1]
    df_dx = math.cos(1/2*x**2 - 1/4*y**2 + 3) * x * math.cos(2*x + 1 - math.exp(
        math.sin(1/2*x**2 - 1/4*y**2 + 3) * math.sin(2*x + 1 - math.exp(y)))
    df_dy = math.cos(1/2*x**2 - 1/4*y**2 + 3) * (-1/2*y) * math.cos(2*x + 1 - ma
        math.sin(1/2*x**2 - 1/4*y**2 + 3) * math.sin(2*x + 1 - math.exp(y)))
    return [df_dx, df_dy]

# --- 1. DESCENSO DEL GRADIENTE Y GRÁFICO 2D ---
print("Iniciando Descenso del Gradiente...")

# Preparamos datos para el mapa de contorno
X_vals = np.linspace(-rango, rango, resolucion)
Y_vals = np.linspace(-rango, rango, resolucion)
Z_vals = np.zeros((resolucion, resolucion))
for ix, x in enumerate(X_vals):
    for iy, y in enumerate(Y_vals):
        Z_vals[iy, ix] = f([x, y])

# Configuramos el gráfico 2D
plt.figure(figsize=(8, 6))
plt.contourf(X_vals, Y_vals, Z_vals, resolucion)
plt.colorbar()

# Ejecutamos el algoritmo
P = P_inicial.copy() # Usamos una copia para no perder el original
plt.plot(P[0], P[1], "o", c="white", label="Inicio")

for _ in range(iteraciones):
    grad = df(P)
    P[0], P[1] = P[0] - TA*grad[0], P[1] - TA*grad[1]
    plt.plot(P[0], P[1], "o", c="red", markersize=2) # Puntos del camino

# Punto final
plt.plot(P[0], P[1], "o", c="green", label="Fin")
plt.legend()
plt.title("Descenso del Gradiente (Vista 2D)")
plt.show()

print(f"Solución encontrada: {P}")
print(f"Valor de la función: {f(P)}")
print("-" * 30)

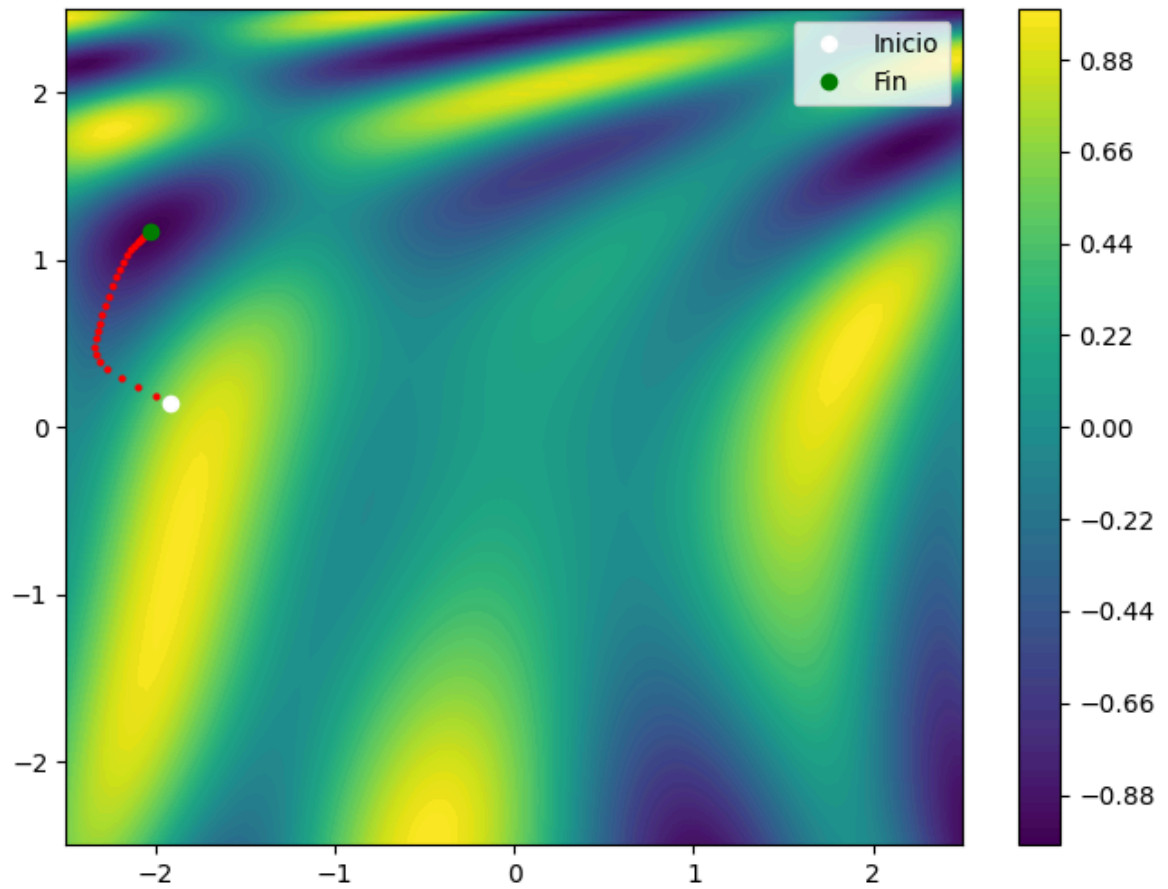
# --- 2. GRÁFICO 3D (Estilo SymPy) ---
print("Generando vista 3D...")
x, y = symbols('x y')
# Función simbólica para SymPy
f_simbolica = sin(1/2 * x**2 - 1/4 * y**2 + 3) * cos(2*x + 1 - exp(y))

# Renderizamos el gráfico 3D
p = plot3d(f_simbolica,
    (x, -rango, rango), (y, -rango, rango),
    title='Superficie de la Función (Vista 3D)',
    size=(10, 10), show=False)
p.show()

```

Iniciando Descenso del Gradiente...

Descenso del Gradiente (Vista 2D)



Solución encontrada: [-2.027650617826627, 1.1718268363570794]

Valor de la función: -1.0

Generando vista 3D...

Superficie de la Función (Vista 3D)

